EventBook: Serverless Event Booking on AWS

Project report

Item	Details
Student	Sameer Naik
Course	AWS
GitHub	github.com/Sameern10
AWS region	US East (Ohio), us-east-2
Scope	Built entirely in the AWS Console. DevOps (CI/CD, Infrastructure as Code) is out of scope.

1. Overview

EventBook is a fully serverless web application where users create an account, browse upcoming events, and book a seat. After a booking, the user receives a confirmation email and the admin receives an alert. A nightly job emails reminders for the next day's events. There are no servers to set up or manage, and every part is an AWS managed service.

2. Objectives

- Build a complete application using AWS serverless services only.
- Learn how Lambda, API Gateway, DynamoDB and S3 work together as the core of an application.
- Add authentication (Cognito), queueing (SQS), workflows (Step Functions), notifications (SNS), email (SES) and scheduling (EventBridge).
- Host the site on S3 with CloudFront, and understand logging and permissions with CloudWatch and IAM.

3. Architecture

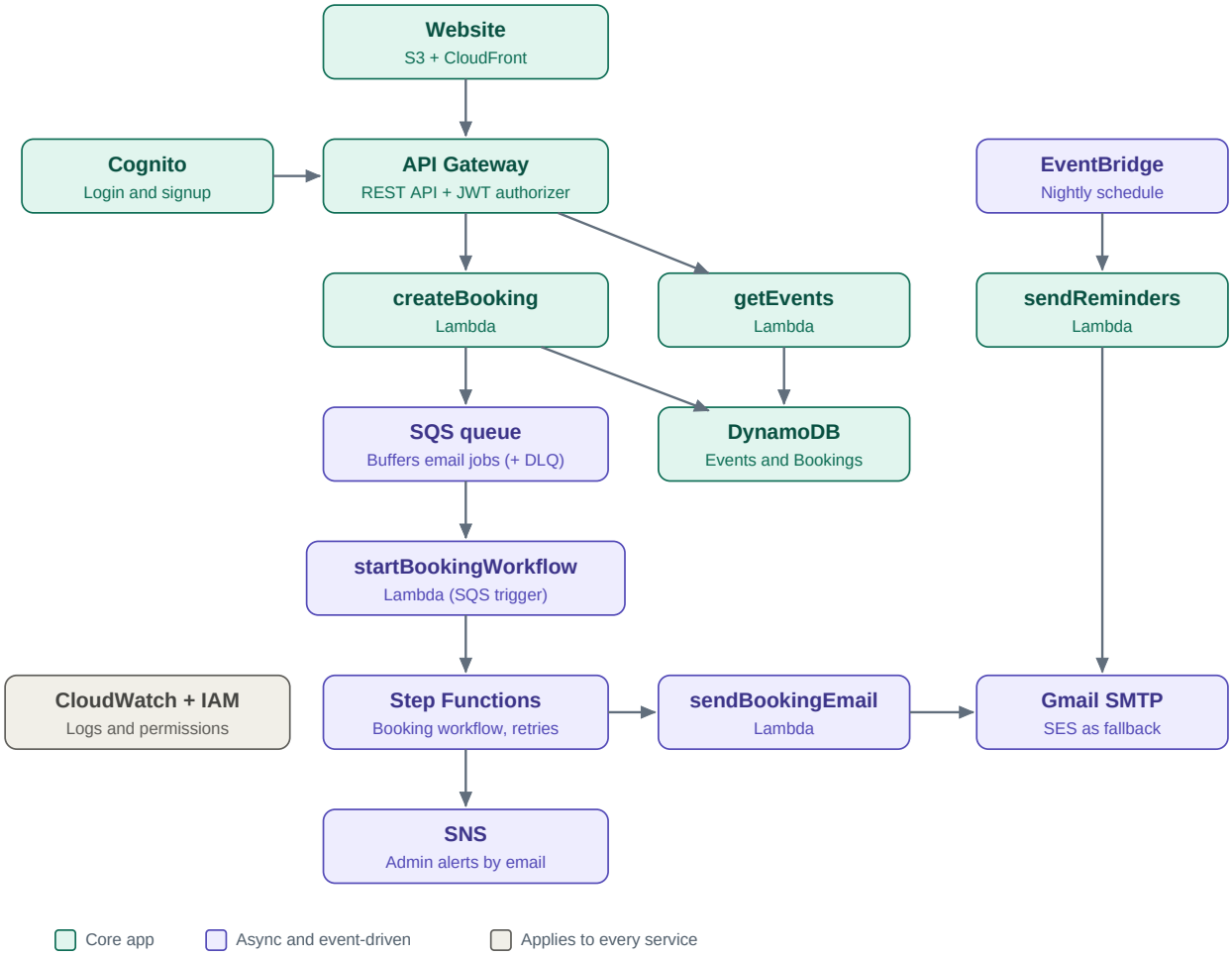


Figure 1: Architecture of EventBook.

4. How a booking works

1. The user logs in through Cognito on the website, which is hosted on S3 and served by CloudFront.
2. Clicking Book now calls API Gateway. A JWT authorizer checks the Cognito token before the createBooking Lambda runs.
3. createBooking reduces the seat count in DynamoDB only if seats are left (so an event can't be overbooked), saves the booking, and puts an email job in an SQS queue. The user gets an immediate response.
4. startBookingWorkflow reads the queue and starts a Step Functions workflow.
5. The workflow calls sendBookingEmail with up to 3 retries, then publishes an SNS alert to the admin. If the email still fails, the admin receives a failure alert instead.
6. Every evening, EventBridge triggers sendReminders, which emails everyone who booked an event happening the next day.

5. AWS services used

Service	Purpose in this project
S3 + CloudFront	Hosts the website. The bucket is private and served only through CloudFront.
Cognito	Sign up, email verification, login, and JWT tokens.
API Gateway	HTTP API: GET /events (public) and POST /bookings (protected by the Cognito authorizer).
Lambda	All backend logic in Python: getEvents, createBooking, startBookingWorkflow, sendBookingEmail, sendReminders.
DynamoDB	Events and Bookings tables, on-demand capacity.
SQS + dead-letter queue	Buffers email jobs so a slow email never slows a booking. Failed messages move to the dead-letter queue after 3 tries.
Step Functions	Booking workflow with retries and error handling.
SNS	Email alerts to the admin about new bookings and failures.
EventBridge Scheduler	Runs the nightly reminder job.
SES	Fallback email sender.
CloudWatch, IAM	Logs for every Lambda, and permissions for every service.

6. Frontend

The website uses plain HTML, CSS and JavaScript, split into index.html, style.css and app.js, with no framework or build step. It shows events as ticket-style cards with category filters and search, a login and sign-up popup, and a booking button that shows the result of each booking.

7. How emails are sent

New AWS accounts start in the SES sandbox, where SES only sends to addresses that were verified one by one. Production access needs a verified domain, which was not available for this project. To let any registered user receive a confirmation, the Lambda sends through Gmail SMTP using an app password stored as an environment variable, and falls back to SES if Gmail fails. A production system would use SES in production mode with a verified domain and DKIM, and keep credentials in AWS Secrets Manager.

8. Screenshots

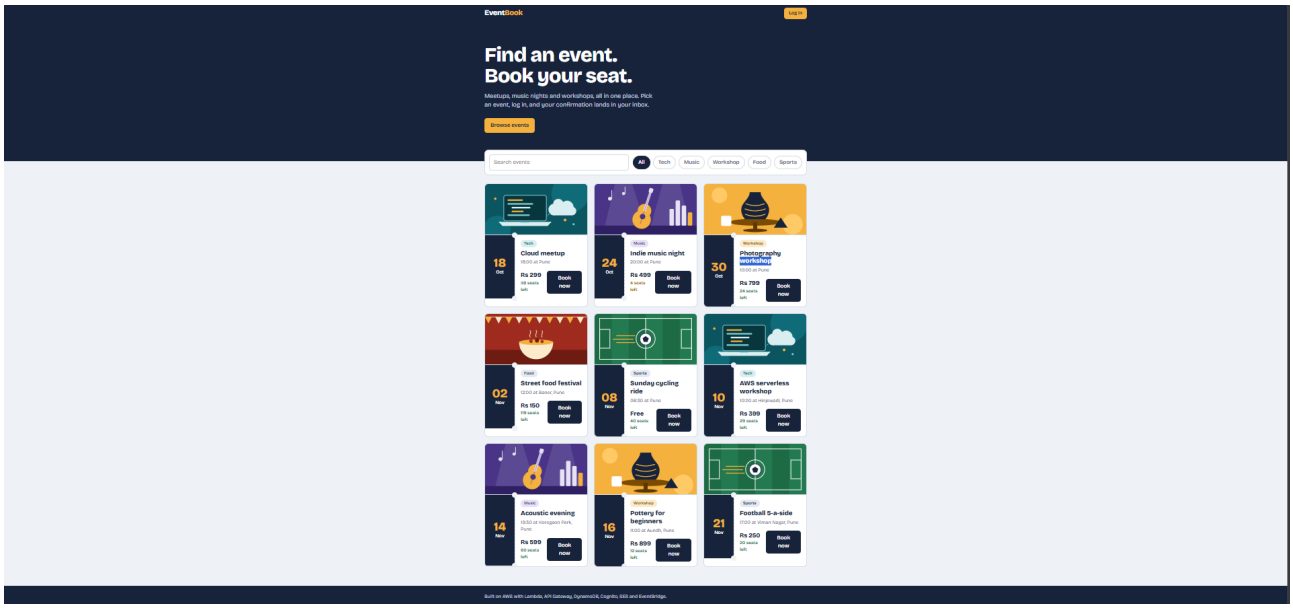


Figure 2: The EventBook website with category filters and search.

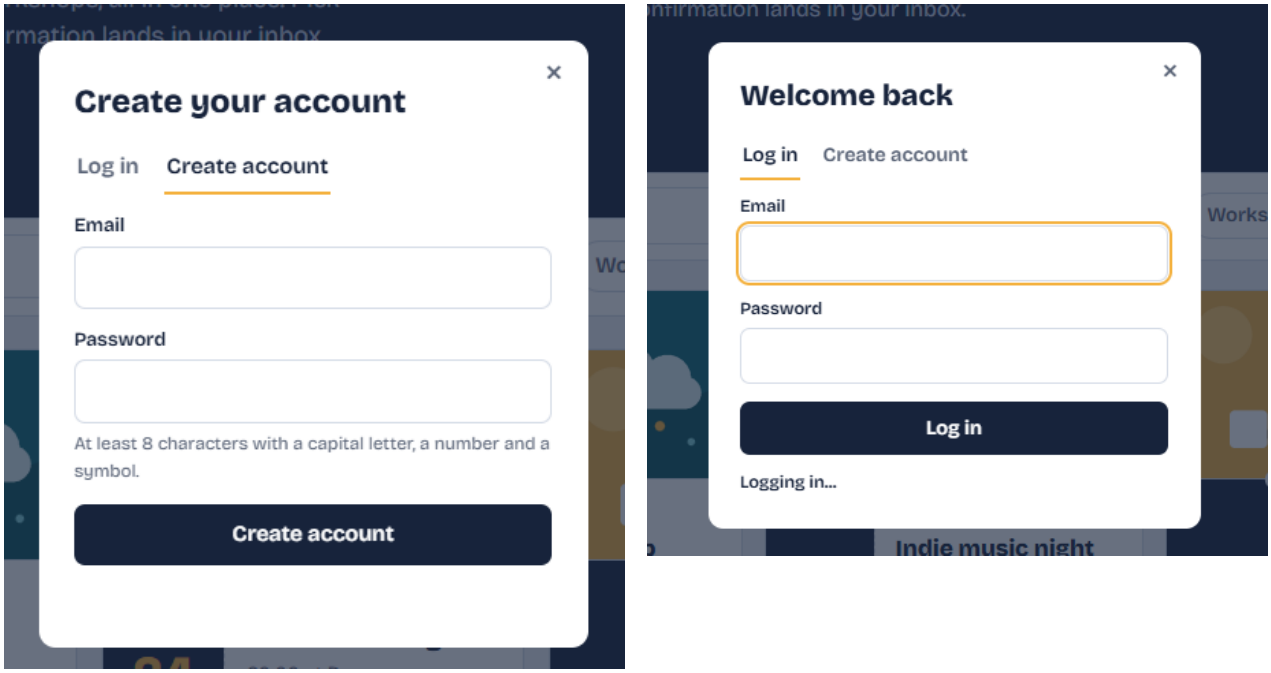


Figure 3: Create account (left) and log in (right), powered by Cognito.



Figure 4: Booking confirmation email received in the inbox.

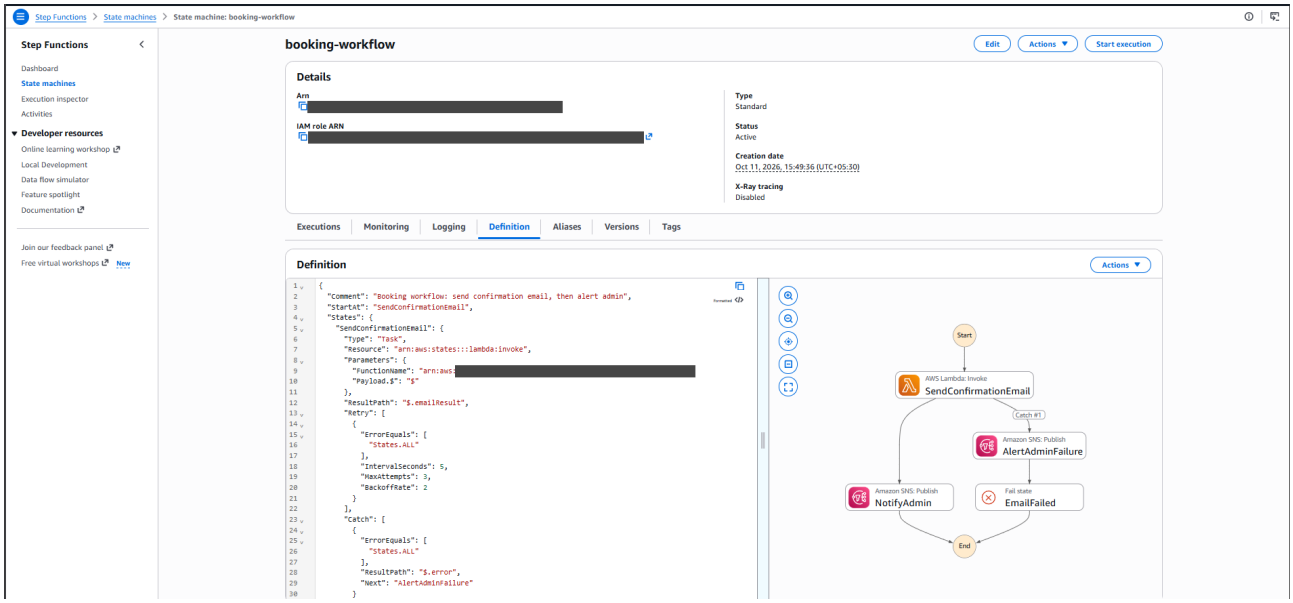


Figure 5: The booking workflow in Step Functions (definition and graph).

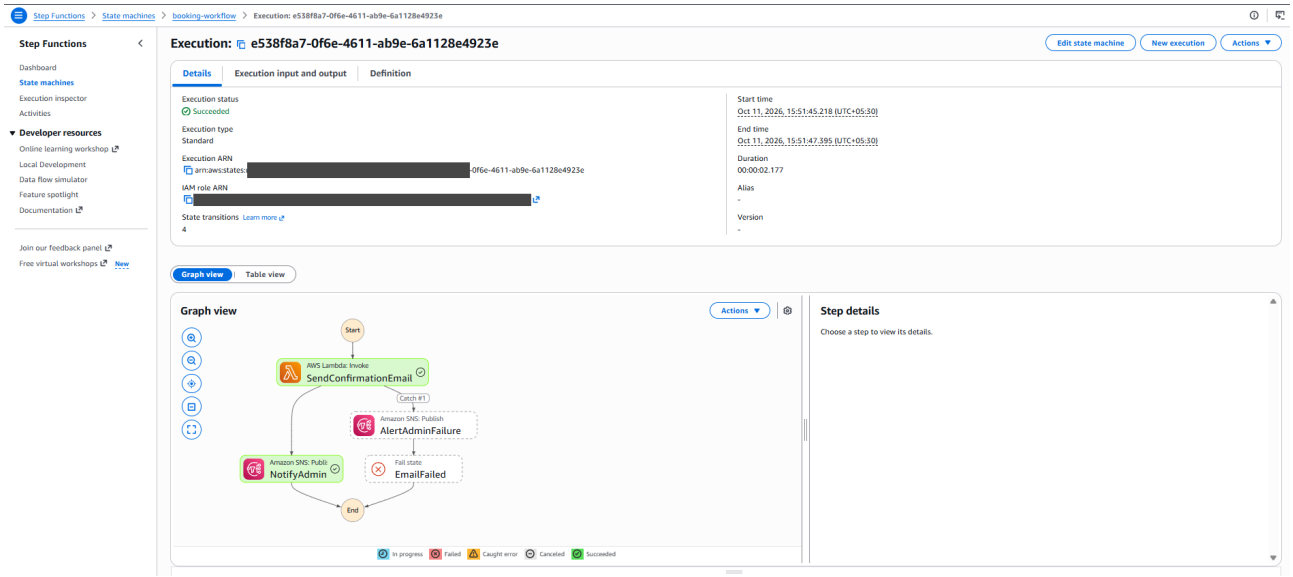


Figure 6: A successful workflow execution.

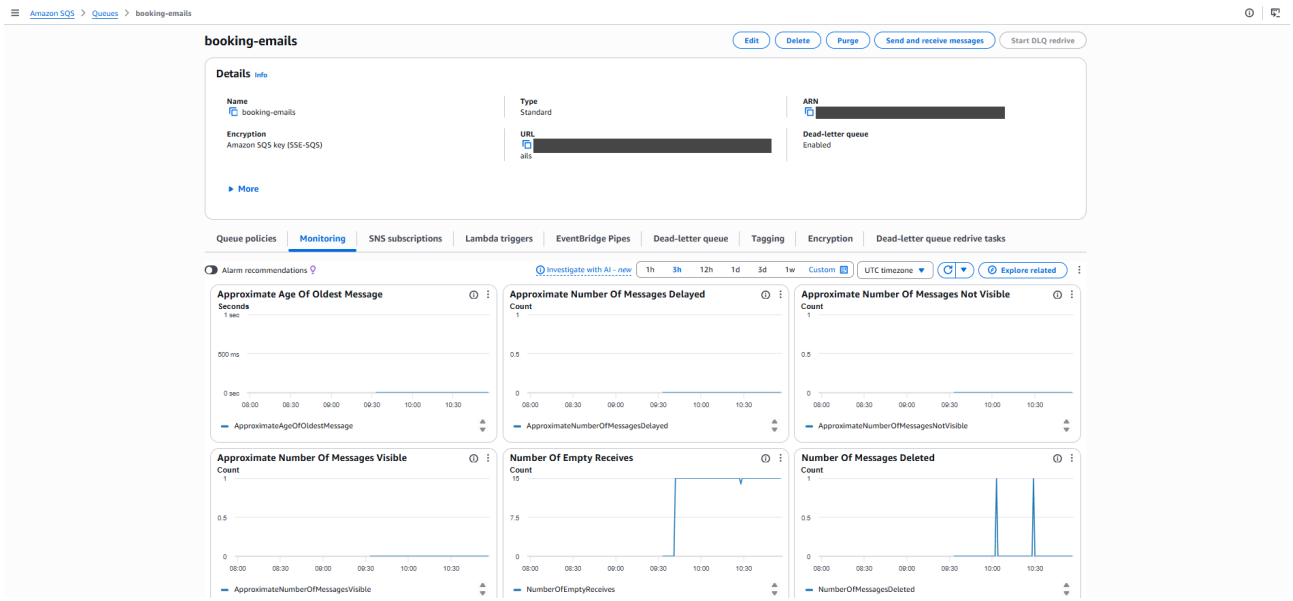


Figure 7: The booking-emails SQS queue with monitoring graphs.

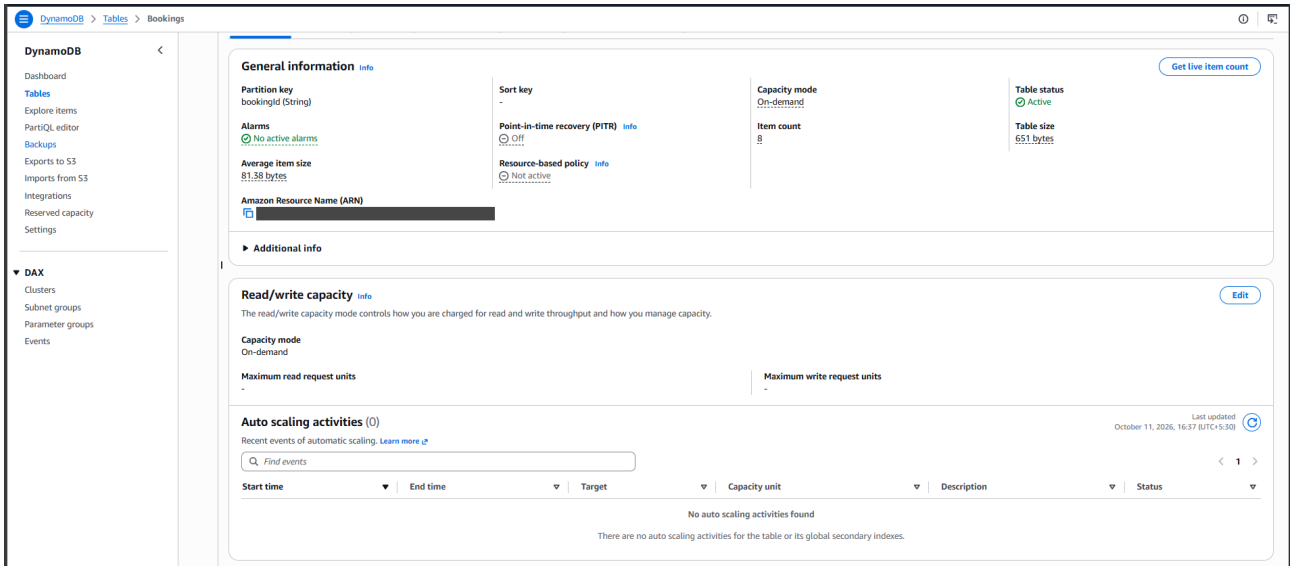


Figure 8: The Bookings table in DynamoDB.

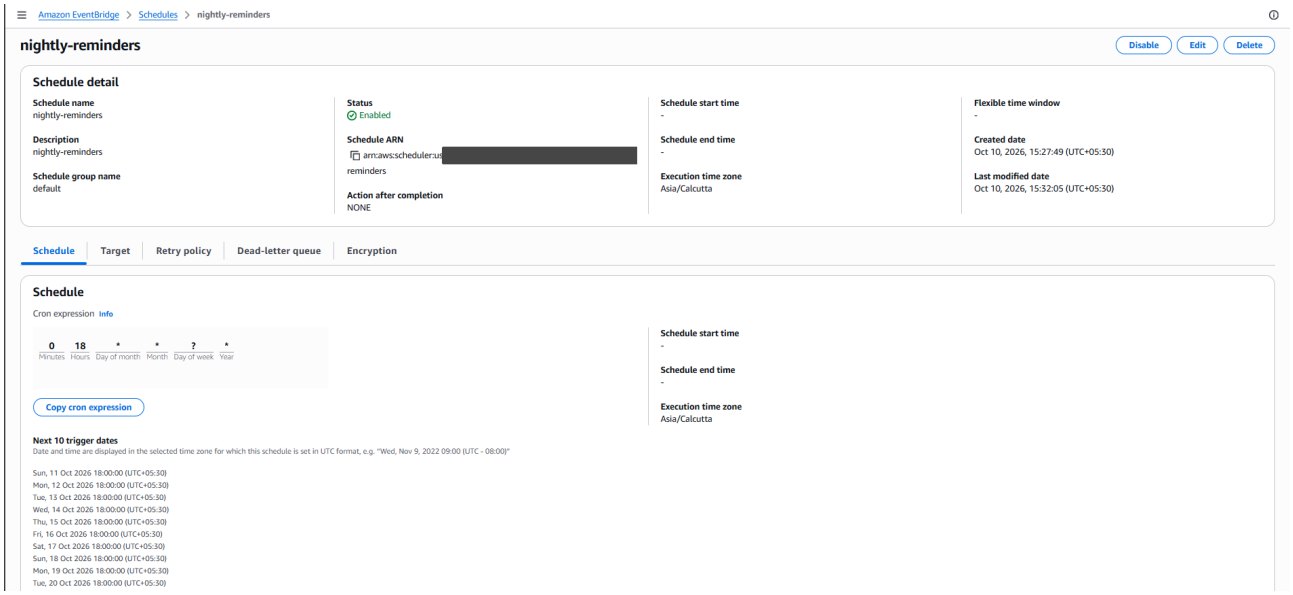


Figure 9: The nightly-reminders schedule in EventBridge.

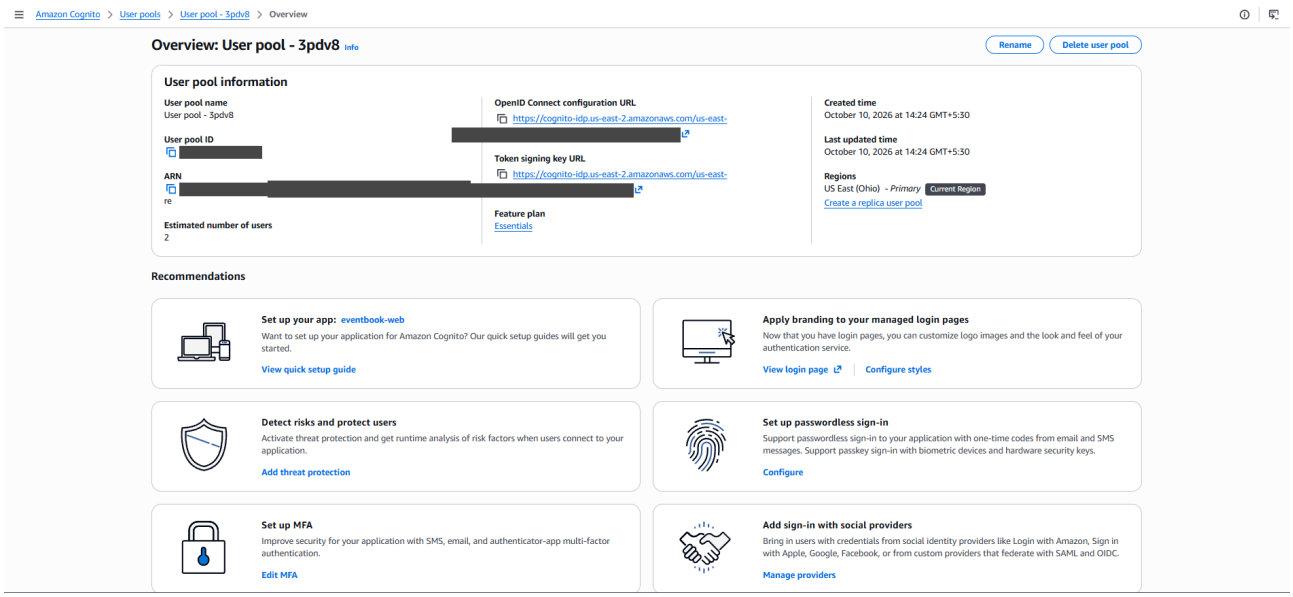


Figure 10: The Cognito user pool.

9. Limitations and future improvements

- Move the Gmail credentials into AWS Secrets Manager, and use SES with a verified domain in production mode.
- Add a forgot-password flow, a My bookings page, and an admin page for creating events and uploading posters to S3.
- Use a global secondary index on the event date instead of scanning the tables for reminders.
- Add API throttling and a budget alert, and restrict CORS to the CloudFront domain.
- Define the infrastructure as code (for example AWS SAM or Terraform) once DevOps is in scope.

10. Outcome

A working, fully serverless booking application that uses all the services above together, built in the AWS Console. The code, workflow definition and setup guide are in the GitHub repository.